

Data Cleaning

Part 2: Cleaning 2024 General Election Data

Load Libraries

```
library(tidyverse)
library(readxl)
```

Read Raw Data

```
# Read the raw precinct-level vote data
raw_data <- read_csv("data/g24_sov_by_g24_svprec.csv", show_col_types = FALSE)

# Display data structure
cat("Raw data dimensions:", nrow(raw_data), "rows x", ncol(raw_data),
    "columns\n")
```

Raw data dimensions: 51123 rows x 76 columns

Inspect Data Quality

```
# Check for suppressed values (marked with "**")
# According to Statewide Database, values are suppressed for privacy when
# counts are very small

# Find vote columns (congressional races)
vote_cols <- names(raw_data)[grep("^CNG", names(raw_data))]
cat("Congressional vote columns found:", length(vote_cols), "\n")
```

Congressional vote columns found: 5

```
print(vote_cols)
```

```
[1] "CNGDEM01" "CNGDEM02" "CNGIND01" "CNGREP01" "CNGREP02"
```

```
# Check for "**" in vote columns
check_suppressed <- function(x) {
  sum(grepl("\\*\\*\\*", as.character(x), fixed = TRUE))
}
```

```

}

suppressed_counts <- sapply(raw_data[vote_cols], check_suppressed)
total_suppressed <- sum(suppressed_counts)
cat("\nTotal suppressed values:", total_suppressed, "\n")

```

Total suppressed values: 0

```

if (total_suppressed > 0) {
  cat("Columns with suppressed values:\n")
  print(suppressed_counts[suppressed_counts > 0])
}

```

Data Cleaning Steps

```

# Clean the data
clean_data <- raw_data %>%
  # Convert "***" to NA in all vote columns
  mutate(across(all_of(vote_cols), ~if_else(. == "***", NA_character_, .)))
%>%
  # Convert vote columns to numeric
  mutate(across(all_of(vote_cols), as.numeric)) %>%
  # Ensure district column is numeric
  mutate(CDDIST = as.numeric(CDDIST)) %>%
  # Ensure precinct key is character
  mutate(SVPREC_KEY = as.character(SVPREC_KEY))

cat("Cleaned data dimensions:", nrow(clean_data), "rows x", ncol(clean_data),
    "columns\n")

```

Cleaned data dimensions: 51123 rows x 76 columns

Validate Data

```

# Check for duplicate precinct keys
dup_check <- clean_data %>%
  count(SVPREC_KEY) %>%
  filter(n > 1)

if (nrow(dup_check) > 0) {
  cat("⚠ Warning:", nrow(dup_check), "duplicate precinct keys found\n")
  print(head(dup_check))
} else {

```

```
cat("✓ All precinct keys are unique\n")
}
```

✓ All precinct keys are unique

```
# Check district assignments
cat("\nDistricts found:", length(unique(na.omit(clean_data$CDDIST))), "\n")
```

Districts found: 53

```
cat("Counties found:", length(unique(clean_data$COUNTY)), "\n")
```

Counties found: 57

```
cat("Total precincts:", nrow(clean_data), "\n")
```

Total precincts: 51123

```
# Check for missing values in key columns
cat("\nMissing values in key columns:\n")
```

Missing values in key columns:

```
cat(" SVPREC_KEY:", sum(is.na(clean_data$SVPREC_KEY)), "\n")
```

SVPREC_KEY: 0

```
cat(" CDDIST:", sum(is.na(clean_data$CDDIST)), "\n")
```

CDDIST: 0

Save Cleaned Data

```
# Write cleaned data to CSV
write_csv(clean_data, "data/g24_sov_by_g24_svprec_clean.csv")
cat("\n✓ Cleaned data saved to: data/g24_sov_by_g24_svprec_clean.csv\n")
```

✓ Cleaned data saved to: data/g24_sov_by_g24_svprec_clean.csv

Part 5: Processing AB 604 Data

Approach B: Proportional Allocation

We'll use census blocks to allocate votes from SR precincts to the new AB 604 districts.

Step 1: Load SR Precinct Vote Data

```
# Read SR precinct voting data
sr_votes <- read_csv("data/state_g24_sov_data_by_g24_srprec.csv",
  show_col_types = FALSE)

cat("SR precinct data:", nrow(sr_votes), "rows\n")
```

SR precinct data: 25245 rows

```
# Find congressional vote columns
sr_vote_cols <- names(sr_votes)[grep("^CNG", names(sr_votes))]
cat("Congressional vote columns:", length(sr_vote_cols), "\n")
```

Congressional vote columns: 5

```
# Clean SR vote data (convert "***" to NA and make numeric)
sr_votes <- sr_votes %>%
  mutate(across(all_of(sr_vote_cols), ~if_else(. == "***", NA_character_, .)))
  %>%
  mutate(across(all_of(sr_vote_cols), as.numeric))

cat("✓ SR votes cleaned\n")
```

✓ SR votes cleaned

Step 2: Load SR Precinct to Block Correspondence

```
# Read the SR precinct to census block mapping
# Each row represents a (potentially fractional) census block
sr_to_block <- read_csv("data/state_g24_sr_blk_map.csv", show_col_types =
FALSE)

cat("SR-to-block mapping:", nrow(sr_to_block), "rows\n")
```

SR-to-block mapping: 421137 rows

```
cat("Columns:", paste(names(sr_to_block), collapse = ", "), "\n")
```

Columns: COUNTY, BLOCK_KEY, FIPS, SRPREC, TRACT, SRPREC_KEY, ELECTION, GEO_TYPE, BLOCK, BLKREG, SRTOTREG, PCTSRPREC, BLKTOTREG, PCTBLK

```
# Key columns:
# BLOCK_KEY: Full census block GEOID
# SRPREC_KEY: SR precinct key
# BLKREG: Number of registered voters on this (fractional) block
# PCTSRPREC: Percent of SR precinct's voters on this block
# SRTOTREG: Total registered voters in the SR precinct
```

Step 3: Load AB 604 Block Assignments

```
# Read AB 604 district assignments by census block
ab604_blocks <- read_csv("data/ab604_block_assignments.csv",
col_names = c("BLOCK_KEY", "AB604_DISTRICT"),
show_col_types = FALSE)

cat("AB 604 block assignments:", nrow(ab604_blocks), "blocks\n")
```

AB 604 block assignments: 519723 blocks

```
cat("New districts:", length(unique(ab604_blocks$AB604_DISTRICT)), "\n")
```

New districts: 52

Step 4: Allocate Votes from SR Precincts to Blocks

```
# Join SR votes with block mapping
# Then allocate votes proportionally based on PCTSRPREC
block_votes <- sr_to_block %>%
```

```
# Join with SR precinct votes
left_join(sr_votes, by = "SRPREC_KEY") %>%
# For each vote column, allocate proportionally to the block
mutate(across(all_of(sr_vote_cols),
               ~. * (PCTSRPREC / 100),
               .names = "{.col}_allocated"))

cat("Block-level votes calculated:", nrow(block_votes), "rows\n")
```

Block-level votes calculated: 421137 rows

Step 5: Aggregate Blocks to AB 604 Districts

```
# Join with AB 604 district assignments
block_votes_with_district <- block_votes %>%
  left_join(ab604_blocks, by = "BLOCK_KEY")

# Check for blocks without district assignment
missing_district <- sum(is.na(block_votes_with_district$AB604_DISTRICT))
cat("Blocks without AB 604 district:", missing_district, "\n")
```

Blocks without AB 604 district: 11

```
# Aggregate to district level
ab604_allocated_cols <- paste0(sr_vote_cols, "_allocated")

ab604_results <- block_votes_with_district %>%
  filter(!is.na(AB604_DISTRICT)) %>%
  group_by(AB604_DISTRICT) %>%
  summarise(across(all_of(ab604_allocated_cols),
                    ~sum(., na.rm = TRUE)),
            .groups = "drop") %>%
# Remove "_allocated" suffix from column names
rename_with(~str_remove(., "_allocated"), all_of(ab604_allocated_cols))

cat("\nAB 604 district results:", nrow(ab604_results), "districts\n")
```

AB 604 district results: 52 districts

Step 6: Save AB 604 Results

```
# Save the AB 604 district-level results
write_csv(ab604_results, "data/ab604_district_results.csv")
cat("✓ AB 604 results saved to: data/ab604_district_results.csv\n")
```

✓ AB 604 results saved to: data/ab604_district_results.csv

Summary

```
cat("\n===== \n")
```

=====

```
cat("DATA CLEANING COMPLETE\n")
```

DATA CLEANING COMPLETE

```
cat("===== \n\n")
```

=====

```
cat("Files created:\n")
```

Files created:

```
cat(" 1. data/g24_sov_by_g24_svprec_clean.csv (2024 actual results)\n")
```

1. data/g24_sov_by_g24_svprec_clean.csv (2024 actual results)

```
cat(" 2. data/ab604_district_results.csv (AB 604 simulated results)\n\n")
```

2. data/ab604_district_results.csv (AB 604 simulated results)

```
cat("Ready for analysis in subsequent parts.\n")
```

Ready for analysis in subsequent parts.